Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 19
Maximum Flow (Part 2)
Μέγιστη Ροή (Μέρος 2)

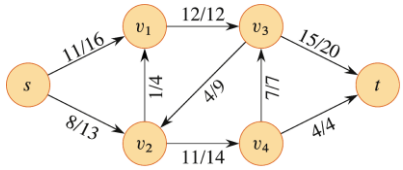
Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, E. D. Demaine, and M. Christodoulakis

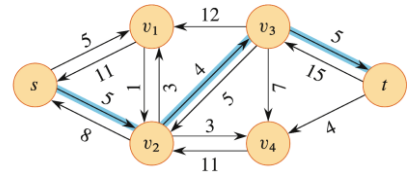
1

The Ford-Fulkerson Method

- Depends on three important notions
  - **Residual network** (**Υπολειπόμενο δίκτυο**)
  - **Augmenting path** (**Αυξητική διαδρομή**)
  - **Cut** (**Τομή**)
- Main Idea:
  - Start with $f(u, v) = 0$ for all $u, v \in V$
  - At each iteration, increase the flow by finding an augmenting path in a residual network
  - Repeat until no augmenting path can be found



Flow Network



Residual Network

2

Cuts

- A **cut** (**τομή**) is a partition of V into S and $\mathrm{T} = V - S$, such that the source $s \in S$ and the sink $t \in T$
- The **net flow** (**καθαρή ροή**) $f(S,T)$ through the cut is the sum of flows $f(u,v)$, where $u \in S$ and $v \in T$

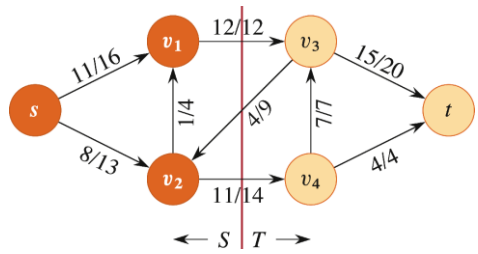
$$f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u)$$

- The **capacity** (**χωρητικότητα**) $c(S,T)$ of the cut is the sum of capacities $c(u,v)$, where $u \in S$ and $v \in T$

$$c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v)$$

- **Minimum cut** (**ελάχιστη τομή**) is a cut with the smallest capacity over all cuts

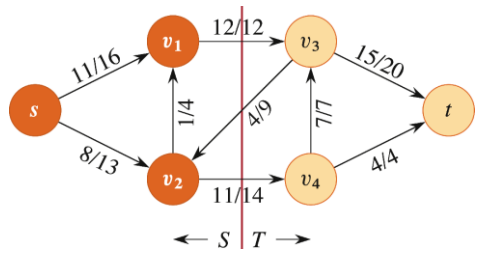
Cuts – Example



- Net flow of cut
  - $f(v_1,v_3) + f(v_2,v_4) - f(v_2,v_3) = 12 + 11 - 4 = 19$
- Capacity of cut
  - $c(v_1,v_3) + c(v_2,v_4) = 12 + 14 = 26$
- Note: Net flow includes flow going both ways across the cut, capacity goes only from S to T

Flow of a Cut

- **Lemma:** The net flow across a cut (S, T) is $f(S, T) = |f|$, where f is the flow of the network
- **Corollary:** The value of any flow f of G is bounded from above by the capacity of any cut (S, T) of G, $|f| \leq c(S, T)$



Max-flow Min-cut Theorem

- Correctness of Ford-Fulkerson comes from this theorem
- **Theorem** (**Max-flow Min-cut Theorem**)

  If f is a flow in G, the following conditions are equivalent:

  1. f is a maximum flow in G
  2. The residual network G_f contains no augmenting paths
  3. $|f| = c(S, T)$ for some cut (S, T) of G
- **Proof**
  - $(1) \Rightarrow (2)$

    Assume f is maximum flow in G, but G_f contains augmenting path p. Then, $f \uparrow f_p$ is a flow with value $> |f|$. Contradiction, since we assumed f was maximum

Max-flow Min-cut Theorem

- **Proof** (cont'd)
  - $(2) \Rightarrow (3)$

    No augmenting path ⇒ no path from s to t in G_f

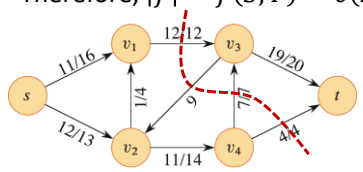
    Define a cut (S, T) where

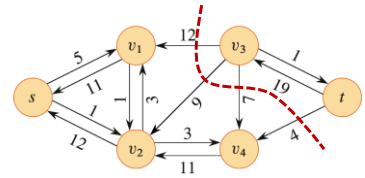
    $S = \{u \in V$: there exists a path from s to u in $G_f\}$, $T = V - S$

    Since no path from s to $t \Rightarrow t \notin S \Rightarrow (S, T)$ is a proper cut

    For vertices $u \in S$ and $v \in T$, we have $f(u, v) = c(u, v)$ otherwise $(u, v) \in E_f$ implying that v should be in S

    Therefore, $|f| = f(S, T) = c(S, T)$



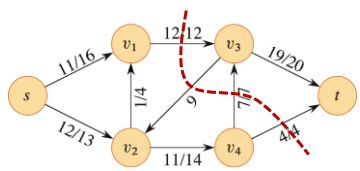


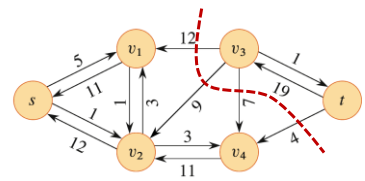
Max-flow Min-cut Theorem

- **Proof** (cont'd)
  - $(3) \Rightarrow (1)$

    We know that $|f| \leq c(S, T)$ for all cuts (S, T)

    Therefore, $|f| = c(S, T)$ implies that f is a maximum flow





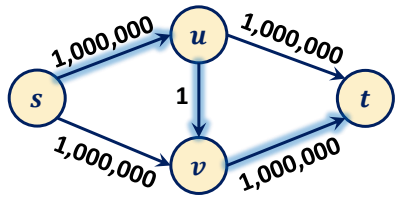
Basic Ford-Fulkerson Algorithm

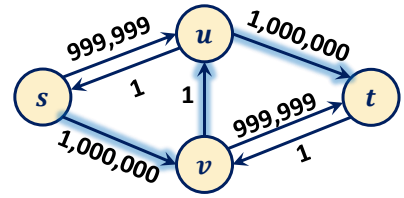
```
FORD-FULKERSON(G, s, t)
1  for each edge (u, v) ∈ G.E
2      (u, v).f = 0
3  while there exists a path p from s to t in the residual network G_f
4      c_f(p) = min{c_f(u, v) : (u, v) is in p}
5      for each edge (u, v) in p
6          if (u, v) ∈ E
7              (u, v).f = (u, v).f + c_f(p)
8          else (v, u).f = (v, u).f − c_f(p)
```


Analysis of Ford-Fulkerson

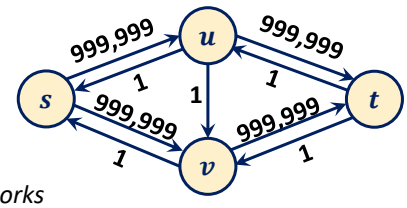
- The running time depends on how the augmenting path p is determined
  - $O(V + E') = O(E)$ time to find a path using BFS or DFS, where $E' = \{(u, v): (u, v) \in E \text{ or } (v, u) \in E\}$
  - *Why?* The flow network is a connected graph $E \geq V - 1$
- If capacities are all integers, then each augmenting path raises $|f|$ by ≥ 1. If max flow is f^*, then need $\leq |f^*|$ iterations ⇒ **Time complexity: $O(E|f^*|)$**
- Running time **not** polynomial in input size!

Ford-Fulkerson Worst-case Example





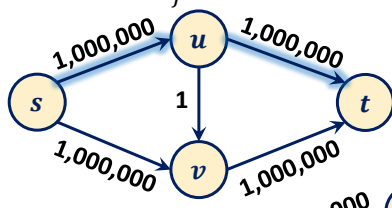
- Takes $O(E|f^*|)$ time with $|f^*| = 2{,}000{,}000$
- 2,000,000 augmentations!

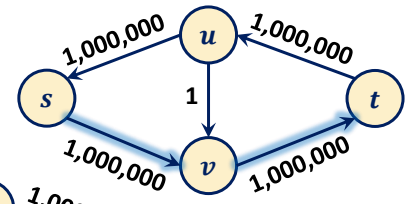


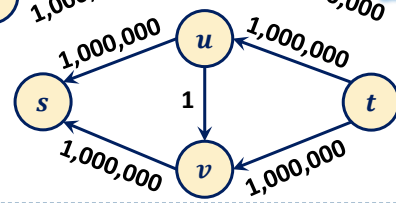
Note: all graphs shown are residual networks

Edmonds-Karp Algorithm

- Edmonds-Karp algorithm
  - Same as Ford-Fulkerson but compute augmenting paths by BFS on residual network G_f. Augmenting paths are **shortest path** $s \rightsquigarrow t$ in G_f in terms of number of edges!





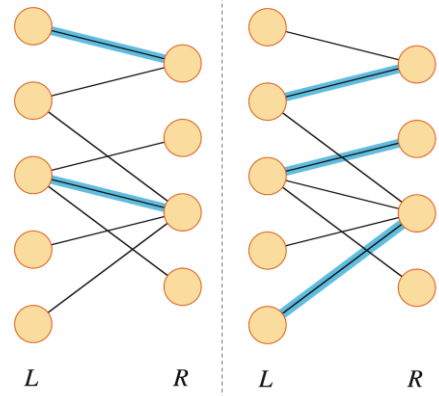


Edmonds-Karp Algorithm

- Edmonds-Karp algorithm
  - Same as Ford-Fulkerson but compute augmenting paths by BFS on residual network G_f. Augmenting paths are **shortest path** $s \rightsquigarrow t$ in G_f in terms of number of edges!
- **Theorem:** The running time is $O(VE^2)$
- Sketch of proof:
  - **Lemma:** The length of the shortest path increases monotonically with each flow augmentation
  - **Theorem:** The total number of flow augmentations performed by Edmonds-Karp is $O(VE)$

Maximum Bipartite Matching

- A graph $G = (V, E)$ is called **bipartite** (**διμερής**) if V can be divided into two parts L and R such that every edge in E is between a vertex in L and a vertex in R
- A **matching** (**αντιστοίχιση**) is a subset M of E, such that for all vertices v in V, at most one edge of M is incident on v
- A **maximum matching** is a matching of maximum **cardinality** (**πληθικότητα**)



Solving Maximum Bipartite Matching

- Reduce the problem to a maximum flow one and solve using the Ford-Fulkerson method
- Form the corresponding flow network $G' = (V', E')$ of the bipartite graph $G = (V, E)$:
  - Add a source vertex s and edges from s to L
  - Direct the edges in E from L to R
  - Add a target vertex t and edges from R to t
  - Assign a capacity of 1 to all edges
- Assume each $v \in V$ has at least one incident edge
- Note that $|E'| = \Theta(\mathrm{E})$
  - *Why?* $|E'| = |\mathrm{E}| + |\mathrm{V}| \leq |\mathrm{E}| + 2|\mathrm{E}| \leq 3|\mathrm{E}|$

Analysing Maximum Bipartite Matching

- **Lemma:** If M is a matching in G, then there is an integer-valued flow f in G' with value $|f| = |M|$ and vice versa
- **Lemma:** The cardinality of a maximum matching M in a bipartite graph equals the value of a maximum flow f in its corresponding flow network G'
- Total running time: $O(\mathrm{E}' f^*) = O(EV)$ (*Why?* $f^* \leq V/2$)

